Assignment_5_Introduction_to_Machine_Learning

2026-08

1) The CARET package has a number of built in data sets including a cell body segmentation data set published by Hill, LaPan, Li, and Haney in 2007. This data set consists on 119 imaging measurements on 2019 cells. These measurements were generated using High Content Screening (HCS), a method used to capture images of cells and draw boundaries around cell bodies. These boundaries are then used to aid in identifying cell types. In particular, this data set reports measurements of cells from the SK-BR-3 cell line and indicates whether each of the cells imaged were poorly segmented (PS) or well segmented (WS).

a) Read in the segmentation data set and check for missing data.
b) Split this data into a training and testing data set. The original study split the data using half of the data for training and the other half for testing so let's go ahead and follow their lead. Split the data in half, use one half for training and the other half for testing.
c) Train a random forest repeated 3-fold cross validation model that repeats 5 times. Make sure that "Class" is the target variable. Use all of the columns in your training data as predictors.

```
# EXPLANATION
# 1) Load packages needed for the assignment
# 2) Load 'segmentationData' data set from caret package and check for missing
#    values
# 3) Split data 50/50 into training and testing sets
# 4) Set up repeated 3-fold cross-validation with 5 repeats
# 5) Train using random forest to predict Class from all other columns

# Read in packages needed for this assignment
packages <- c("RANN", "tidyverse", "caret", "remotes", "kernlab")

for (pkg in packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
  library(pkg, character.only = TRUE)
}

# remotes::install_version("AppliedPredictiveModeling", version = "1.2")

# Load in sample dataset from caret package
data(segmentationData)
raw_data <- segmentationData

# Checking for na values in the initial data set (0 = none missing)
sum(is.na(raw_data))
```

```
## [1] 0
```

```r
# Set seed for repeatable results
set.seed(1121)

# Split 50/50 based on class
training_index <- createDataPartition(raw_data$Class, p = 1/2,
                                      list = FALSE,
                                      times = 1)

raw_training_data <- raw_data[training_index,]

testing_data <- raw_data[-training_index,]

# Verify sampling adheres to specified proportion (half the data for each set)
dim(raw_training_data)
```

```
## [1] 1010   61
```

```r
dim(testing_data)
```

```
## [1] 1009   61
```

```r
dim(raw_data)
```

```
## [1] 2019   61
```

```r
# Use cross-validation with 3 folds repeated 5 times for 15 model fits
fitcontrol <- trainControl(method = "repeatedcv",
                           number = 3,
                           repeats = 5)

# Use random forest to evaluate every other column as a predictor
model_1 <- train(Class ~ .,
                         data = raw_training_data,
                         method = "rf",
                         trControl = fitcontrol)

# Check results
model_1
```

```
## Random Forest
##
## 1010 samples
##   60 predictor
##    2 classes: 'PS', 'WS'
##
## No pre-processing
## Resampling: Cross-Validated (3 fold, repeated 5 times)
## Summary of sample sizes: 673, 674, 673, 673, 674, 673, ...
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
```

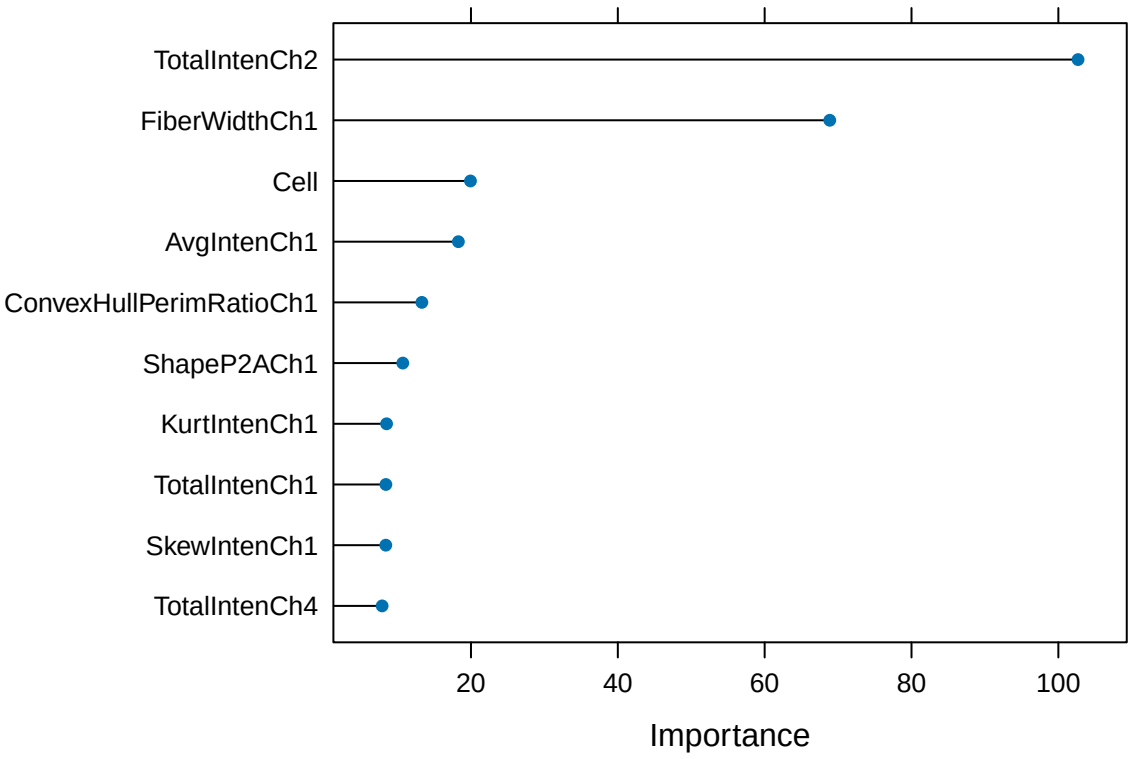
```
##    2    0.8174285  0.5940082
##   31    0.8239614  0.6140321
##   60    0.8243600  0.6152770
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 60.
```

2) In the previous question I asked you to train a random forest model on all of the variables in your data set. We do not necessarily require feature selection with random forest models because random forest models inherently preform a type of feature selection. However,feature selection can help us eliminate noisy or redundant features, thereby reducing computational burden. Run the same model again, but this time only include the top 10 variables as predictors.

```r
# EXPLANATION
# 1) Calculate variable importance from model_1
# 2) Plot and sort the 10 most important scores
# 3) Save the top 10 variable names
# 4) Retrain the random forest using only the 10 most important predictors

# Calculate the importance of variables used in model_1
important_variables <- varImp(model_1, scale = FALSE)

# Plot the 10 most important variables
plot(important_variables, top = 10)
```



```r
# Sort the variables by importance and store them
top_10_variables <- important_variables$importance %>%
  arrange(desc(Overall)) %>%
  head(10)

# Check the top 10 variables sorted by importance
top_10_variables
```

```
##                            Overall
## TotalIntenCh2           102.681646
## FiberWidthCh1            68.868005
## Cell                     19.933655
## AvgIntenCh1              18.287158
## ConvexHullPerimRatioCh1  13.309966
## ShapeP2ACh1              10.717470
## KurtIntenCh1              8.505203
## TotalIntenCh1             8.410445
## SkewIntenCh1              8.401572
## TotalIntenCh4             7.898341
```

```r
# Store the row names of the top 10 most important variables
top_vars <- rownames(top_10_variables)

# Retrain the model using only the top 10 most important variables
model_2 <- train(Class ~ .,
                 data = raw_training_data[, c("Class", top_vars)],
                 method = "rf",
                 trControl = fitcontrol)
# Check results
model_2
```

```
## Random Forest
##
## 1010 samples
##   10 predictor
##    2 classes: 'PS', 'WS'
##
## No pre-processing
## Resampling: Cross-Validated (3 fold, repeated 5 times)
## Summary of sample sizes: 673, 673, 674, 673, 673, 674, ...
## Resampling results across tuning parameters:
##
##   mtry  Accuracy   Kappa
##    2    0.8348517  0.6357009
##    6    0.8366345  0.6413526
##   10    0.8291113  0.6261500
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 6.
```

3) Using the model you trained above, generate predictions on the testing set. After running your model, create a confusion matrix visualization in `ggplot2`. Report the accuracy of your predictions in the main

title of your plot. Make sure you have included appropriate x- and y-axis titles! Write your confusion matrix out to a .png file and upload it with your assignment submission. Hint: Here is some starter code to learn how to generate a visualization like this: https://www.energycode.org/posts/09032022-confusion-matrix/.

```
# EXPLANATION
# 1) Predict class for the testing set using model_2
# 2) Create a confusion matrix comparing predictions to true classes
# 3) Extract the accuracy as a percentage for the plot title
# 4) Plot the confusion matrix with ggplot
# 5) Save the plot as a .png for the submission

# Test the model on data that it's not trained on
predictions <- predict(model_2, newdata = testing_data)

# rows = predicted, columns = actual, diagonal = correct predictions
conf_mat <- confusionMatrix(predictions, testing_data$Class)
conf_mat
```

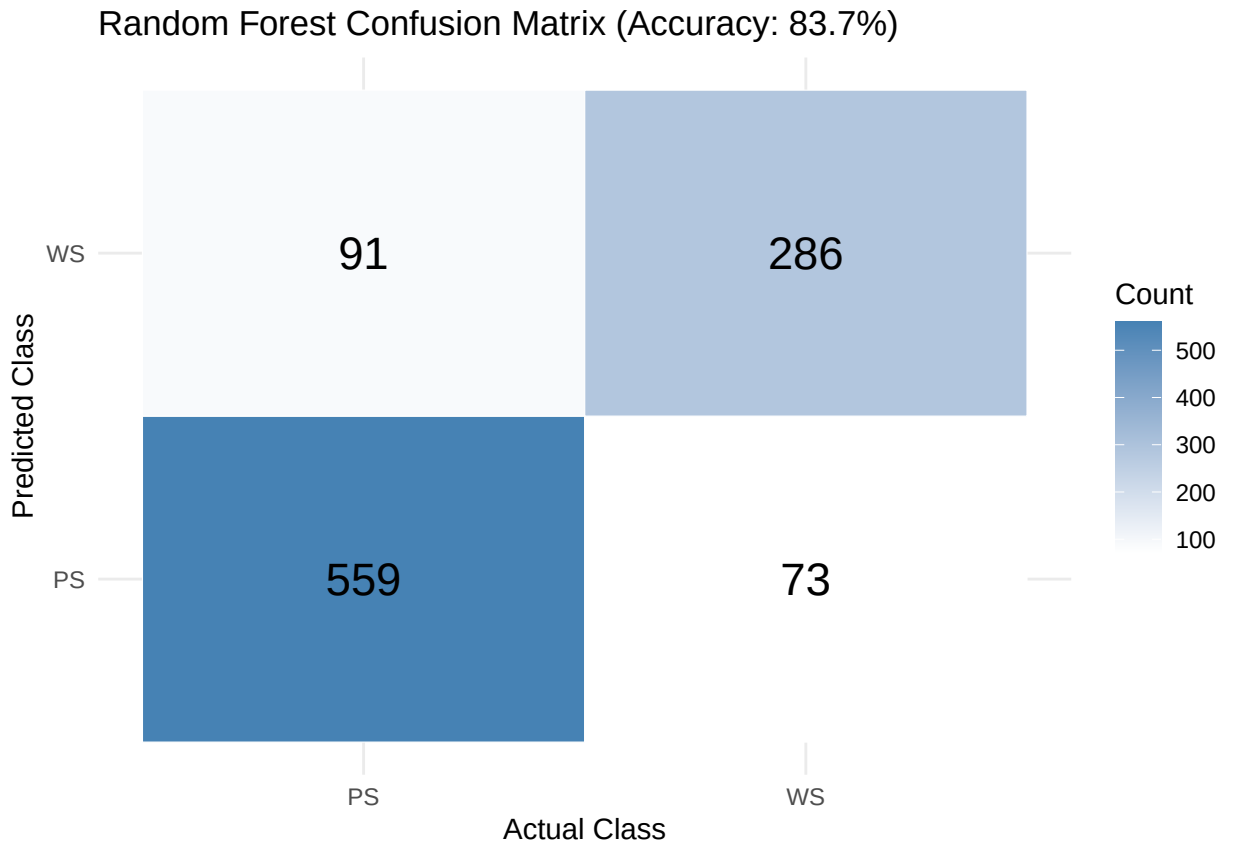
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  PS  WS
##         PS 559  73
##         WS  91 286
##
##                Accuracy : 0.8375
##                  95% CI : (0.8132, 0.8597)
##     No Information Rate : 0.6442
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.6494
##
##  Mcnemar's Test P-Value : 0.1844
##
##             Sensitivity : 0.8600
##             Specificity : 0.7967
##          Pos Pred Value : 0.8845
##          Neg Pred Value : 0.7586
##              Prevalence : 0.6442
##          Detection Rate : 0.5540
##    Detection Prevalence : 0.6264
##       Balanced Accuracy : 0.8283
##
##        'Positive' Class : PS
##
```

```
# Round the accuracy to decimal for the plot title
accuracy_title <- round(conf_mat$overall["Accuracy"] * 100, 1)

# Convert the table to a data frame so ggplot can create the visualization
conf_mat_df <- as.data.frame(conf_mat$table)
```

```r
# Create the plot with titles and labels
conf_mat_plot <- ggplot(conf_mat_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 6) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(title = paste0("Random Forest Confusion Matrix (Accuracy: ", accuracy_title, "%)"),
       x = "Actual Class",
       y = "Predicted Class",
       fill = "Count") +
  theme_minimal()

# Check the plot
conf_mat_plot
```




```r
# Save the plot as a .png
ggsave("Confusion_Matrix.png", plot = conf_mat_plot, width = 6, height = 5, dpi = 300)
```

4) Train a support vector machine model with 3-fold cross validation. Make sure that "Class" is the target variable. Use all of the columns in your training data as predictors. Make sure to indicate that your metric is ROC, as this is a binary classification. Since SVMs are sensitive to the scale of features include "center" and "scale" in your `preProcess` parameter when you train your model.

```r
# EXPLANATION
# 1) Set up a 3-fold cross-validation
```

```r
# 2) Train a radial SVM predicting class from all other columns
# 3) Center and scale predictors to account for distance
# 4) Select the best tuning by ROC
# 5) Review cross-validated ROC, sensitivity, and specificity

# Set up a 3-fold cross-validation
svm_control <- trainControl(method = "cv",
                            number = 3,
                            classProbs = TRUE,
                            summaryFunction = twoClassSummary)

# Train model_3 as a radial SVM
model_3 <- train(Class ~ .,
                 data = raw_training_data,
                 method = "svmRadial",
                 metric = "ROC",
                 preProcess = c("center", "scale"),
                 trControl = svm_control)

# Check the results
model_3
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 1010 samples
##   60 predictor
##    2 classes: 'PS', 'WS'
##
## Pre-processing: centered (60), scaled (60)
## Resampling: Cross-Validated (3 fold)
## Summary of sample sizes: 673, 674, 673
## Resampling results across tuning parameters:
##
##   C     ROC        Sens       Spec
##   0.25  0.8771576  0.8661390  0.6944444
##   0.50  0.8837727  0.8784704  0.6861111
##   1.00  0.8885812  0.8815640  0.7027778
##
## Tuning parameter 'sigma' was held constant at a value of 0.01380502
## ROC was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.01380502 and C = 1.
```

5) Generate predictions for your support vector machine model. Like we did with the random forest model, make a confusion matrix visualization. Save the visualization to a .png file and submit it along with this assignment. Make sure to include an appropriate main title with the model accuracy as well as any x- and y- axis titles you think are important!

```r
# EXPLANATION
# 1) Predict class for the testing set using model_3 (SVM)
# 2) Create a confusion matrix comparing predictions to known classes
# 3) Extract accuracy as a % for the plot title
# 4) Plot the confusion matrix with ggplot
# 5) Save the plot as a .png for the submission
```

```r
# Test the model on data that it's not trained on
svm_predictions <- predict(model_3, newdata = testing_data)

# rows = predicted, columns = actual, diagonal = correct predictions
svm_conf_mat <- confusionMatrix(svm_predictions, testing_data$Class)
svm_conf_mat
```

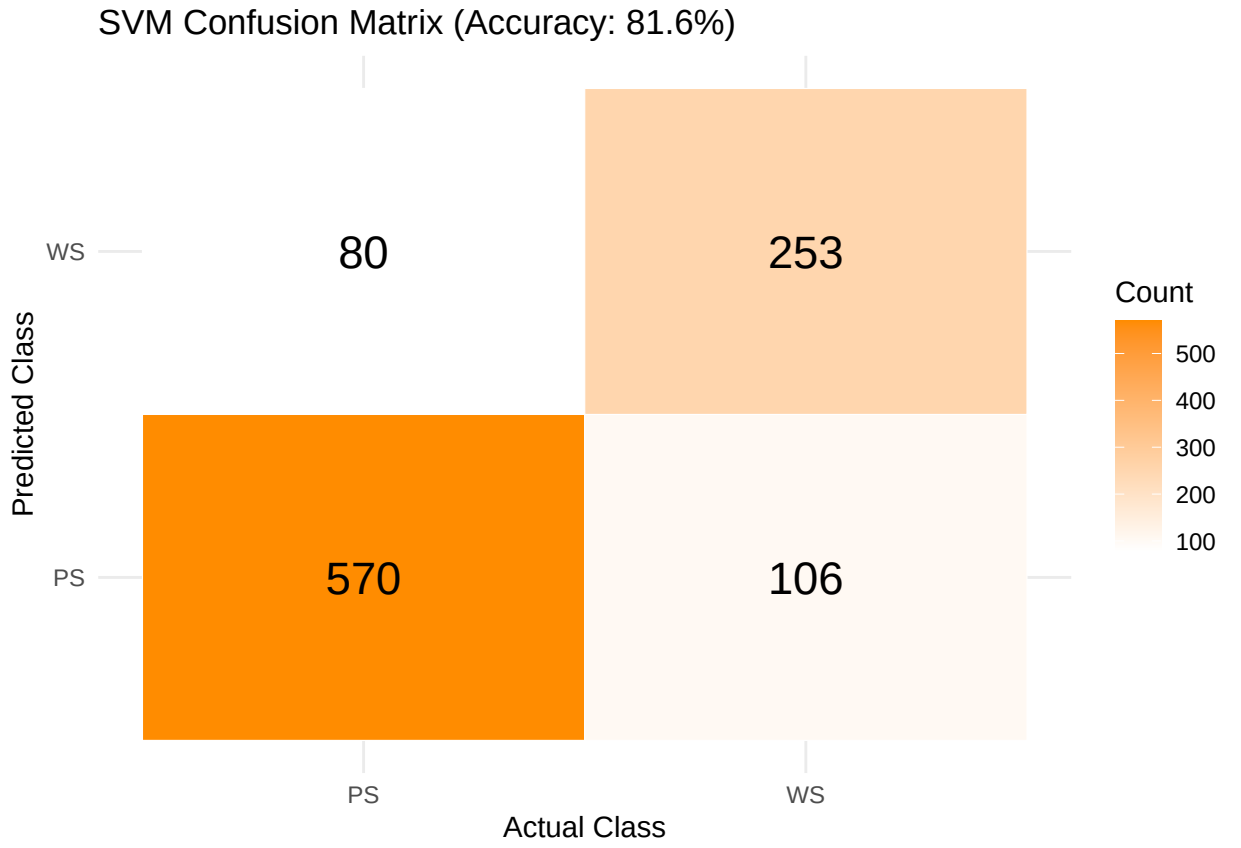
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  PS  WS
##         PS 570 106
##         WS  80 253
##
##                Accuracy : 0.8157
##                  95% CI : (0.7903, 0.8391)
##     No Information Rate : 0.6442
##     P-Value [Acc > NIR] : < 2e-16
##
##                   Kappa : 0.5912
##
##  Mcnemar's Test P-Value : 0.06679
##
##             Sensitivity : 0.8769
##             Specificity : 0.7047
##          Pos Pred Value : 0.8432
##          Neg Pred Value : 0.7598
##              Prevalence : 0.6442
##          Detection Rate : 0.5649
##    Detection Prevalence : 0.6700
##       Balanced Accuracy : 0.7908
##
##        'Positive' Class : PS
##
```

```r
# Round the accuracy to decimal for the plot title
svm_accuracy <- round(svm_conf_mat$overall["Accuracy"] * 100, 1)

# Convert the table to a data frame so ggplot can create the visualization
svm_conf_df <- as.data.frame(svm_conf_mat$table)

# Create the plot with titles and labels
svm_plot <- ggplot(svm_conf_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 6) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  labs(title = paste0("SVM Confusion Matrix (Accuracy: ", svm_accuracy, "%)"),
       x = "Actual Class",
       y = "Predicted Class",
       fill = "Count") +
  theme_minimal()
```

```
# Check the plot
svm_plot
```




```
# Save the plot as a .png
ggsave("svm_confusion_matrix.png", plot = svm_plot, width = 6, height = 5, dpi = 300)
```